

# Compliance Buddy

Your Automated Robot Security Engineer

*"It finds security holes in your code, fixes them, and submits the fix for your approval — all by itself."*

This document explains how Compliance Buddy works in plain, everyday language. No programming background needed. If you can understand how an airport security check works, you can understand this.

|                            |                                                                        |
|----------------------------|------------------------------------------------------------------------|
| <b>What it does</b>        | Automatically finds and fixes security weaknesses in Java code         |
| <b>Who uses it</b>         | Development teams who want security issues fixed without manual effort |
| <b>How fast</b>            | From bug found to fix ready in under 10 minutes                        |
| <b>Human effort needed</b> | One click: Approve or Reject the suggested fix                         |
| <b>AI used</b>             | GPT-4o (OpenAI) as primary, Claude Sonnet (Anthropic) as backup        |
| <b>Version</b>             | v2 — June 2026 (9 components, 4 major new features)                    |

# Section 1 — What Problem Does It Solve?

---

## The Old Way (Without Compliance Buddy)

Imagine your codebase is like a large building. A security inspector (SonarQube) walks through it every day and leaves Post-it notes wherever it finds problems — an unlocked door, a broken window, a missing fire extinguisher.

In the old way, a developer has to:

- Read every Post-it note manually
- Research how to fix each type of problem
- Write the fix themselves
- Test that the fix doesn't break anything
- Submit the fix for a team review
- Wait for approval
- Repeat for every new problem found

This process can take hours or days per issue, and security bugs pile up faster than teams can fix them.

## The New Way (With Compliance Buddy)

### The short version

Compliance Buddy reads those Post-it notes, figures out the fix, applies it, tests it, and hands it to a human for a single yes/no approval — automatically, 24 hours a day.

### Real-world analogy

Think of Compliance Buddy like a robot auto-mechanic at a dealership. The car's onboard computer flags a fault. The robot mechanic reads the error code, looks up the right fix in its repair manual, orders the part, installs it, test-drives the car, then parks it in the "ready for final inspection" bay. The human service manager does one final check and signs off. That's it.

## Section 2 — The Big Picture: Architecture Diagram

The diagram below shows all the pieces of Compliance Buddy and how they connect. Don't worry about the technical names — the sections that follow will explain each part in plain language.

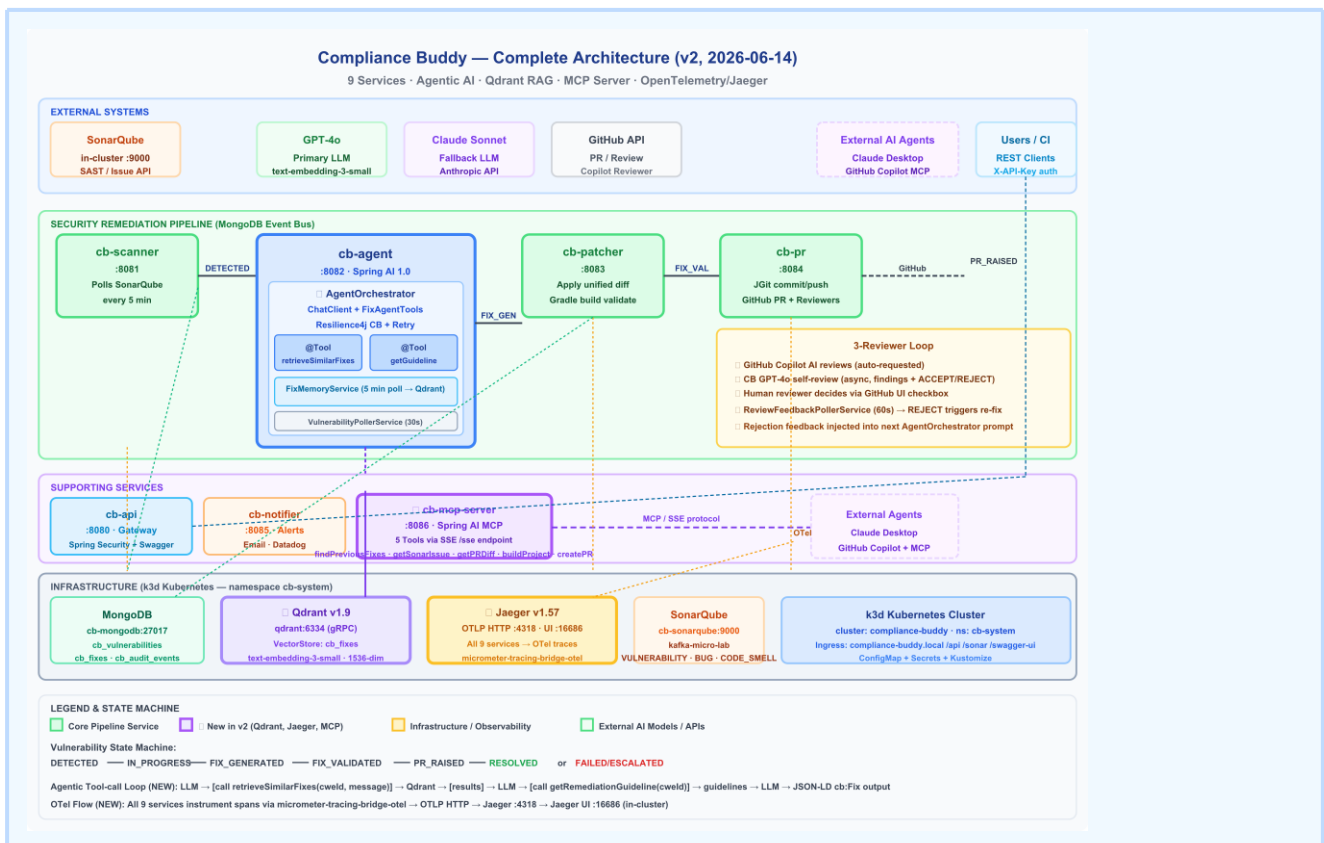


Figure 1 — Compliance Buddy v2 Complete Architecture. Green boxes = the main fixing pipeline. Blue box = the AI brain. Purple = new features. Orange arrows = activity tracking. Grey arrows = users talking to the system.

### How to read the diagram

**Top row (blue band)** — outside systems CB talks to: the code scanner, the AI models, GitHub, and external tools

**Green band (middle)** — CB's core pipeline: scanner → AI brain → patcher → PR creator

**Purple band** — helper services: the API gateway, email alerts, and the new tool server

**Grey band (bottom)** — the filing cabinets and record-keepers: databases and the tracking system

**Arrows with dashes** — optional or background connections

## Section 3 — How It Works, Step by Step

---

Here is the full journey of a security bug, from discovery to fix, explained like a story.

### ① The Scout goes on patrol (every 5 minutes)

A component called **cb-scanner** wakes up every 5 minutes and asks SonarQube: "Found anything new?" SonarQube is the code scanner — it reads all the Java source code looking for security weaknesses like SQL injection, cross-site scripting, and hardcoded passwords. When cb-scanner finds a new problem, it writes it down in the database and stamps it *DETECTED*.

#### Think of it like...

A neighbourhood watch volunteer who walks the streets every 5 minutes and logs any new graffiti or broken windows in a shared notebook that the whole team can see.

### ② The AI Brain wakes up and starts thinking (30 seconds later)

Another component, **cb-agent**, checks the notebook every 30 seconds. When it sees a new DETECTED entry, it picks it up and calls in the AI (GPT-4o — the same model that powers ChatGPT's advanced version). The AI doesn't just guess a fix — it first *asks itself questions*: "Have I seen this type of bug before? What worked last time?" Only after gathering that context does it write the actual code fix.

#### Think of it like...

A doctor who, before prescribing medicine, first looks up your medical history and checks the hospital database for similar cases. Then they write the prescription based on what worked for others, not just a guess.

### ③ The Handyman applies the fix and tests it

**cb-patcher** receives the AI's proposed fix. It carefully applies the changes to the actual code files (like a surgeon making precise incisions — only changing the exact lines that need changing), then runs the full test suite to confirm the code still compiles and builds correctly. If it builds, the fix is stamped *BUILD\_VALIDATED*. If the build breaks, the fix is rejected and the AI tries again.

### ④ The fix gets filed and stored in memory

Once a fix passes the build test, it's stored in two places:

- The regular database (MongoDB) — the full details of what was fixed
- The AI memory (Qdrant — see Section 4) — a "smart" searchable version so future fixes can learn from it

#### ⑤ The Secretary creates a Pull Request on GitHub

**cb-pr** takes the validated fix, creates a new branch in your GitHub repository, commits the changed files, and opens a Pull Request — just like a human developer would. The PR title and description are automatically written to explain what was fixed and why.

#### ⑥ Three reviewers check the fix

Three reviewers look at the PR simultaneously:

- **GitHub Copilot AI** — reviews the code diff and adds comments automatically
- **CB itself (GPT-4o)** — does a second pass, posting numbered findings with an ACCEPT or REJECT checkbox
- **You (the human)** — looks at the PR on GitHub and ticks the checkbox: Accept or Reject

#### ⑦ Approved? Done. Rejected? Try again.

If you tick **Accept**: the vulnerability is marked RESOLVED and the team gets an email. Done.

If you tick **Reject** (with a reason): CB closes the PR, reads your feedback, and tells the AI to try again — this time with your rejection reason baked into the new prompt. CB learns from the rejection to avoid the same mistake.

#### ⑧ Alerts go out

**cb-notifier** sends email alerts to the team when a vulnerability is escalated (tried 3 times without success) or when a PR is merged. It also pushes metrics to Datadog so dashboards stay up to date.

## Section 4 — The 4 New Features (v2 Upgrades)

Version 2 added four major improvements. Here's what changed and why it matters.

### ■ New Feature 1 — AI Memory (Qdrant Vector Store)

■ Before (v1): The AI was "amnesiac" — it started fresh every time with no knowledge of past fixes

■ After (v2): The AI has a searchable memory of every past fix it successfully made

Every time a fix is approved and merged, CB stores it in a special database called **Qdrant**. Qdrant isn't like a normal database where you search by exact words. Instead, it understands *meaning*. When the next similar bug appears, CB asks: "Have I fixed something like this before?" and Qdrant searches by concept — finding related fixes even if the exact wording is different.

#### Think of it like...

A chef's recipe book that not only stores exact recipes but can also recommend "similar dishes" when you tell it what ingredients you have. Ask for "something spicy with chicken" and it surfaces the 5 most relevant recipes — even if you don't remember the dish name.

**Practical result:** Over time, CB gets faster and more accurate because it doesn't reinvent the wheel for the same class of bug. A SQL injection fix it learned in Month 1 helps it write better fixes for SQL injections in Month 6.

### ■ New Feature 2 — Smart AI Agent (Agentic Architecture)

■ Before (v1): The AI received one fixed prompt and gave one answer — no reasoning, no context-gathering

■ After (v2): The AI reasons through the problem step-by-step, using tools to gather context before answering

In v1, the AI was handed a problem and told to answer in one shot — like asking someone to solve a maths problem without letting them use a calculator or look anything up.

In v2, the AI is given **tools it can call**. Before writing the fix, it first runs:

- **Memory check:** "Show me the 5 most similar bugs fixed in the past" (searches Qdrant)
- **Guideline lookup:** "What's the standard approach for this type of vulnerability?" (checks a built-in knowledge base)
- Only then does it write the actual fix, using both sources of information

Think of it like...

The difference between a junior employee who just guesses an answer versus a senior consultant who first pulls the client file, checks industry best practices, and then gives a considered recommendation. Same intelligence, completely different quality of output.

**Safety net:** If the AI tools fail, CB automatically falls back to the simpler approach (like v1) — so the system always produces *some* fix rather than crashing.

■ New Feature 3 — Tool Server for Other AI Agents (MCP Server)

- Before (v1): CB was a closed system — no other AI tool could interact with it
- After (v2): External AI assistants like Claude Desktop or GitHub Copilot can directly use CB's capabilities

CB now runs a small server called **cb-mcp-server** that speaks a standard language called MCP (Model Context Protocol). Other AI assistants that also speak MCP can connect to CB and call its functions like they were built-in tools.

The 5 tools external AI agents can call:

| Tool name         | What it does in plain English                                 |
|-------------------|---------------------------------------------------------------|
| findPreviousFixes | Look up every fix CB has ever made for a specific type of bug |
| getSonarIssue     | Get full details about a specific security warning by its ID  |

|              |                                                                                 |
|--------------|---------------------------------------------------------------------------------|
| getPRDiff    | Show exactly what code changed in a given Pull Request                          |
| buildProject | Kick off a build of the code on a specific branch and report pass/fail          |
| createPR     | Create a Pull Request on GitHub with specified title, description, and branches |

Think of it like...

Instead of CB being a locked room that only CB staff can enter, it now has a reception window where other AI assistants can walk up and say "I need to know about past SQL injection fixes" and CB hands them the information directly.

Concretely: a developer could open Claude Desktop, type "What SQL injection fixes did CB make last month?" — and Claude would call CB's MCP server behind the scenes and return the answer. No web browser, no logging into a portal.

■ New Feature 4 — Activity Tracking Dashboard (OpenTelemetry + Jaeger)

- Before (v1): If something went wrong inside CB, we could only guess where by reading hundreds of log lines
- After (v2): Every action across all 9 components is traced and visualised in a timeline dashboard

CB now uses a standard called **OpenTelemetry** to record timing information for every step it takes. This data flows into **Jaeger**, a dashboard that shows a visual timeline of everything that happened — which component did what, in what order, and how long each step took.

Think of it like...

Imagine a parcel delivery system. OpenTelemetry is the barcode scanner at every depot. Jaeger is the tracking website where you can see "Parcel left warehouse at 09:02, arrived at sorting centre at 09:15, out for delivery at 10:30" — but for every single action CB performs across all its components.

- If a fix takes unusually long, the dashboard shows exactly which step is slow

- If a step fails, the dashboard shows the exact moment and what it was trying to do

- All 9 CB components contribute their timing data to one unified view

# Section 5 — Who's Who: The 9 Components

CB is made up of 9 separate programs that each have one job, like specialists in a hospital.

| Component       | Job Title Analogy          | What it actually does                                                                                        |
|-----------------|----------------------------|--------------------------------------------------------------------------------------------------------------|
| cb-scanner      | Security Inspector         | Checks SonarQube every 5 min, logs new security problems to the shared database                              |
| cb-agent        | AI Doctor + Researcher     | Picks up detected problems, consults its memory + guidelines, and generates a code fix using GPT-4o          |
| cb-patcher      | Surgical Nurse             | Applies the AI's fix to the real code files and runs the build to verify nothing broke                       |
| cb-pr           | Executive Secretary        | Commits the fix to a new branch, opens a GitHub Pull Request, invites reviewers, and polls for your decision |
| cb-api          | Front Desk Receptionist    | The only door to CB from the outside — accepts REST API calls, checks identity (API key), routes requests    |
| cb-notifier     | Team Communicator          | Sends email alerts when a bug can't be fixed after 3 tries, or when a PR is merged                           |
| cb-mcp-server ■ | External Relations Officer | Lets other AI tools (Claude Desktop, GitHub Copilot) call CB's capabilities through the MCP standard         |

|                 |                                |                                                                                                       |
|-----------------|--------------------------------|-------------------------------------------------------------------------------------------------------|
| <b>Qdrant</b> ■ | Institutional Memory / Library | Stores every past fix in a searchable, meaning-aware format so the AI can find similar past solutions |
| <b>Jaeger</b> ■ | CCTV Control Room              | Records and displays a visual timeline of every action across all 9 components in real time           |

---

# Section 6 — The Database: Where Everything Is Stored

CB uses two types of storage, each for a different purpose.

## Storage Type 1 — MongoDB (the Filing Cabinet)

MongoDB stores the structured facts: which vulnerabilities were found, what fixes were attempted, the PR URLs, build results, review decisions, audit logs. You can think of it as a very organised spreadsheet with multiple tabs. Every CB component reads from and writes to this shared filing cabinet.

| Drawer (Collection) | What's stored there                                                                                  |
|---------------------|------------------------------------------------------------------------------------------------------|
| cb_vulnerabilities  | Every security problem ever found, with its current status (DETECTED, FIXED, RESOLVED...)            |
| cb_fixes            | Every fix attempt: the code changes, confidence score, which AI model made it, build result, PR link |
| cb_review_sessions  | The GitHub PR review details, Copilot findings, and whether you ticked Accept or Reject              |
| cb_audit_events     | A full history log of every action — who did what and when                                           |

## Storage Type 2 — Qdrant (the Smart Memory)

Qdrant is a completely different kind of storage. Instead of storing facts as rows and columns, it stores the *meaning* of text as a list of 1,536 numbers (called an "embedding vector"). Each number represents a shade of meaning. Fixes about the same type of bug end up with very similar number lists — so Qdrant can find them even if the exact words differ.

### Why two databases?

MongoDB is fast for exact lookups: "give me fix with ID 12345". Qdrant is fast for fuzzy, conceptual searches: "give me fixes that are semantically similar to this SQL injection in a Spring Boot

repository controller". They do completely different jobs.

## Section 7 — Security and Safety Nets

---

CB handles code that goes into production. Several safety nets are in place.

### 1. Nothing gets merged without human approval

Every single fix must pass through the 3-reviewer step and receive a human tick before it can be merged. CB can create PRs, but it cannot merge them. The human is always the final decision-maker.

### 2. Fixes are tested before you ever see them

cb-patcher runs the full Gradle build on every fix before a PR is even opened. If the build fails — even for an unrelated reason — the fix is discarded and the AI tries again. You never see a fix that doesn't compile.

### 3. Rejection feedback improves the next attempt

When you reject a fix and write a reason ("This changes too much", "Use a different approach"), CB reads your reason and feeds it directly into the AI's next attempt. Over time CB learns what your team accepts and what it doesn't.

### 4. Maximum 3 attempts per bug

If CB tries 3 times and all 3 are rejected (or fail the build), it stops trying and sends an escalation alert to the team: "This bug needs human attention — CB couldn't fix it." No infinite loops, no blocking the pipeline.

### 5. Backup AI model

If the primary AI (GPT-4o) is unavailable, CB automatically switches to Claude Sonnet (Anthropic's model) as a backup. A "circuit breaker" mechanism prevents repeated failed calls from overwhelming the AI service.

### 6. All credentials are locked away

API keys for OpenAI, Anthropic, GitHub, and SonarQube are stored in Kubernetes Secrets — never in the code, never in a configuration file that could be accidentally shared.

## Section 8 — A Day in the Life of a Bug

---

Here is a realistic example of how CB handles a SQL injection vulnerability from start to finish.

### The scenario

A developer commits code that builds a database query by gluing strings together instead of using safe parameterised queries. This is called SQL injection — one of the most common and dangerous security vulnerabilities.

#### 09:00 SonarQube finds the bug.

SonarQube's scanner runs and flags: "CWE-89: SQL injection in UserRepository.java, line 47." The issue gets an ID: sqube-issue-abc123.

#### 09:05 cb-scanner picks it up.

On its next 5-minute patrol, cb-scanner fetches new SonarQube issues, sees sqube-issue-abc123, and saves it to MongoDB as a new Vulnerability with status DETECTED.

#### 09:05:30 cb-agent starts working.

cb-agent's 30-second check fires. It picks up the DETECTED vulnerability and calls the AI. The AI first searches Qdrant: "Show me the 5 most similar SQL injection fixes from the past." Qdrant finds 3 relevant past fixes — all using PreparedStatement. The AI also looks up the CWE-89 guideline: "Always use parameterised queries." Now it writes the fix.

#### 09:06 Fix is generated.

The AI produces a precise code change: replace the string concatenation in UserRepository.java with a PreparedStatement. The fix is saved to MongoDB (status: FIX\_GENERATED).

#### 09:06:30 cb-patcher tests the fix.

cb-patcher picks up the FIX\_GENERATED record, applies the code change to the actual file, and runs `./gradlew build`. Build passes in 4 minutes. Status → BUILD\_VALIDATED. The fix is also stored in Qdrant for future memory.

#### 09:11 GitHub PR is opened.

cb-pr creates branch `cb-fix/cwe89-abc123`, commits the change, and opens PR #247 "Security fix: CWE-89 SQL Injection in UserRepository.java". GitHub Copilot is requested as a reviewer. CB's

own AI review is kicked off in parallel.

### **09:12** Reviews come in.

GitHub Copilot posts 2 inline comments about the code. CB posts a structured review: "No security findings. Fix correctly uses PreparedStatement. Recommend ACCEPT." with a checkbox.

### **09:30** You click Accept.

You glance at the PR on GitHub, read the change, and tick the Accept checkbox in CB's review comment. Within 60 seconds, cb-pr detects your tick and updates MongoDB: vulnerability → RESOLVED. The team receives an email: "Security issue CWE-89 in UserRepository.java resolved via PR #247."

**Total elapsed time: ~30 minutes (mostly waiting for your review)**

Of that 30 minutes, CB's active processing time was under 10 minutes. Your effort: roughly 2 minutes of reading and one click.

## Section 9 — Frequently Asked Questions

---

### Q: Can CB accidentally break my codebase?

No. CB never merges code automatically. It creates Pull Requests that a human must approve first. Additionally, every fix is built and tested before it even appears as a PR. If the build fails, you never see the fix.

### Q: What happens if the AI makes a bad fix?

You reject it. CB reads your rejection reason, learns from it, and tries again with improved instructions. After 3 failed attempts, it stops and alerts the team that the bug needs manual attention.

### Q: Does CB fix ALL types of security bugs?

Currently CB focuses on the most common Java security vulnerabilities flagged by SonarQube (SQL injection, XSS, insecure deserialization, etc.). It improves over time as its fix memory grows.

### Q: Can I see what CB is doing right now?

Yes. The Jaeger dashboard (new in v2) shows a real-time visual timeline of every action. You can also call the REST API at `/api/v1/metrics/dashboard` to get a summary of vulnerability counts by status.

### Q: Is my code sent to OpenAI or Anthropic?

CB sends the vulnerability context and the relevant code snippet to the AI model, not your entire codebase. Only the specific lines flagged by SonarQube are included in the AI prompt.

### Q: What is the MCP server for?

It lets other AI tools (like Claude Desktop or GitHub Copilot Chat) call CB's functions directly. For example, a developer could ask their AI assistant: "What fixes did CB make this week?" and the AI would use the MCP server to get a live answer from CB — no web browser needed.

### Q: What if GPT-4o is down?

CB automatically switches to Claude Sonnet (Anthropic) as a backup. If both are unavailable, the circuit breaker kicks in to prevent repeated failed calls, and the fix generation is queued for retry.

**Q: How does CB improve over time?**

Every approved fix is stored in Qdrant memory. Next time a similar bug appears, CB searches this memory before writing the fix. The more fixes CB makes, the more context-aware and accurate it becomes. It's like training a new employee who gets better with every project they complete.

## Section 10 — Quick Reference Card

### Status labels you'll see

| Status        | Meaning in plain English                                                    |
|---------------|-----------------------------------------------------------------------------|
| DETECTED      | SonarQube found a bug; CB knows about it but hasn't started fixing it yet   |
| IN_PROGRESS   | The AI is currently working on a fix                                        |
| FIX_GENERATED | The AI has written a proposed fix; waiting for build validation             |
| FIX_VALIDATED | The fix passed the build test; waiting for a PR to be opened                |
| PR_RAISED     | A Pull Request is open on GitHub; waiting for your review decision          |
| RESOLVED      | You approved the fix; done                                                  |
| FAILED        | 3 attempts were made and all failed or were rejected; needs human attention |
| ESCALATED     | CB has notified the team by email; human action required                    |

### Key numbers

| What                     | How often / How many |
|--------------------------|----------------------|
| SonarQube check          | Every 5 minutes      |
| AI fix check             | Every 30 seconds     |
| PR review polling        | Every 60 seconds     |
| Memory indexing (Qdrant) | Every 5 minutes      |

|                                |                                               |
|--------------------------------|-----------------------------------------------|
| Max fix attempts per bug       | 3 attempts before escalation                  |
| AI models used                 | 2 — GPT-4o (primary) + Claude Sonnet (backup) |
| Total CB components            | 9 (7 services + Qdrant + Jaeger)              |
| Tools exposed via MCP          | 5 tools for external AI agents                |
| Time from bug found to PR open | Under 10 minutes (typical)                    |

**Who is responsible for what**

| Task                        | Who does it                            |
|-----------------------------|----------------------------------------|
| Finding bugs                | SonarQube (automated scanner)          |
| Writing the fix             | GPT-4o / Claude Sonnet AI              |
| Testing the fix             | cb-patcher (Gradle build)              |
| Creating the PR             | cb-pr (automated)                      |
| Code reviewing the fix      | GitHub Copilot AI + CB's own AI review |
| Final approval or rejection | YOU — the human developer              |
| Merging the PR              | YOU — always a human decision          |

**Bottom line**

Compliance Buddy does 95% of the security fix work automatically. Your job is the final 5%: read the PR, decide if the fix looks right, click Accept or Reject. That's it.